

Sistema de Plugins

`org.openmarkov.core`

Documentación técnica de la arquitectura de extensibilidad

Descubrimiento automático · Anotaciones · Managers · Contratos

Módulo: `org.openmarkov.core`

Fecha: 30 de marzo de 2026

*Los diagramas UML se generan a partir de los archivos
img/*.puml mediante PlantUML (véase el Makefile adjunto).*

Índice

1. Introducción	3
1.1. Cuatro pilares del sistema	3
1.2. Cuatro categorías de plugin	3
2. Vista de alto nivel	4
3. Motor de búsqueda de plugins	4
3.1. PluginClassCategory	5
3.2. PluginLoader	5
3.3. PluginSearch	6
3.4. ExtensionTree	6
4. Anotaciones de descubrimiento y de contrato	6
4.1. Anotaciones de descubrimiento	7
4.1.1. @PotentialType	7
4.1.2. @Constraint	7
4.1.3. @NetworkTypeInfo	8
4.1.4. @FormatType	8
4.2. Anotaciones de contrato	8
4.2.1. @ImplementationRequirements	8
5. Managers	9
5.1. PotentialUtils	9
5.2. ConstraintManager	10
5.3. NetworkTypeUtils	10
5.4. FormatManager	11
6. Subsistemas de plugins	11
6.1. Clases base de los puntos de extensión	11
6.2. Subsistema de Potenciales	12
6.2.1. Descubrimiento y herencia de la anotación	12
6.2.2. Filtrado por compatibilidad	13
6.2.3. Indexación interna de TablePotential	13
6.3. Subsistema de Restricciones	13
6.3.1. Interacción con el sistema de ediciones	14
6.4. Subsistema de Tipos de Red	14
6.5. Subsistema de Formatos de Archivo	15
7. Flujos de descubrimiento: diagramas de secuencia	16
7.1. Descubrimiento de potenciales	16
7.2. Construcción de la lista de restricciones	17
7.3. Resolución de formato al abrir un archivo	18
8. Cómo añadir un nuevo plugin	19
8.1. Nuevo Potencial	20
8.2. Nueva Restricción	20
8.3. Nuevo Tipo de Red	21

8.4. Nuevo Formato de Archivo	22
9. Mapa completo del sistema	22
10.Consideraciones de rendimiento y ciclo de vida	23
10.1. Coste de inicialización	23
10.2. Thread safety	23
10.3. Extensibilidad en tiempo de ejecución	23
10.4. Resumen comparativo	24

§1 Introducción

En el contexto de OpenMarkov, un **plugin** es una clase Java que extiende o implementa un tipo base previamente definido en el núcleo del sistema (por ejemplo, un tipo de potencial, una restricción estructural o un formato de archivo), y que el sistema descubre y registra *automáticamente al arrancar*, sin que sea necesario modificar ningún código existente. El desarrollador simplemente anota la nueva clase con la anotación apropiada, la coloca en el classpath y el sistema la incorpora como si siempre hubiera estado ahí.

Este mecanismo está diseñado para soportar la extensibilidad a largo plazo: nuevos módulos de OpenMarkov pueden añadir tipos de red, algoritmos de inferencia o formatos de archivo sin crear dependencias circulares ni modificar el núcleo.

1.1 Cuatro pilares del sistema

El sistema de plugins se sustenta en cuatro mecanismos que trabajan de forma coordinada. En conjunto, proporcionan un ciclo completo de extensión: el desarrollador anota → el motor descubre → el manager registra → el dominio usa.

1. **Anotaciones de descubrimiento** (§4): marcan las clases como puntos de extensión y les asignan metadatos (nombre, comportamiento por defecto, versión, etc.). Son el contrato visible que el desarrollador de un plugin debe cumplir.
2. **Motor de búsqueda de plugins** (§3): escanea el classpath al arrancar y construye un catálogo de todas las clases que cumplen los criterios de cada categoría. Opera de forma transparente, sin que el código de dominio tenga que invocarlo explícitamente.
3. **Managers** (§5): singletons que consumen el catálogo construido por el motor y exponen al resto del sistema una API de alto nivel para obtener instancias de plugins por nombre, filtrarlos por compatibilidad o listar todos los disponibles.
4. **Contrato de implementación** (§4.2): la anotación `@ImplementationRequirements` documenta qué constructores y métodos deben existir en cada tipo de plugin. Los tests de integración verifican que ningún plugin rompa el contrato.

1.2 Cuatro categorías de plugin

Actualmente el núcleo de OpenMarkov define cuatro categorías de plugin, cada una asociada a una anotación de descubrimiento y un manager específico. La tabla siguiente resume su propósito y sus relaciones principales.

Categoría	Anotación	Propósito	Manager
Potencial	<code>@PotentialType</code>	Distribuciones de probabilidad / utilidad	<code>PotentialUtils</code>
Restricción	<code>@Constraint</code>	Reglas estructurales de una red	<code>ConstraintManager</code>
Tipo de red	<code>@NetworkTypeInfo</code>	Define el tipo de modelo probabilístico	<code>NetworkTypeUtils</code>
Formato de arch.	<code>@FormatType</code>	Lectura/escritura de redes en disco	<code>FormatManager</code>

§2 Vista de alto nivel

La figura 2 muestra los cuatro grupos de actores del sistema y sus relaciones principales: las anotaciones marcan los plugins, las clases base los definen estructuralmente, los managers los registran y el motor de búsqueda proporciona la infraestructura de descubrimiento. Los paquetes están ordenados de izquierda a derecha según su papel en el ciclo de descubrimiento: las anotaciones son el punto de entrada del desarrollador; las clases base definen los contratos; los managers son los intermediarios que el dominio usa; y el motor es el mecanismo interno que hace posible todo lo anterior.

Leyenda de colores:

- **Amarillo** — Anotaciones de descubrimiento
- **Naranja** — Clases base de los puntos de extensión
- **Verde** — Managers (singletons de acceso)
- **Cian** — Motor de búsqueda (`org.openmarkov.plugin`)
- **Azul claro** — Implementaciones concretas de tipos de red
- **Rosa** — Implementaciones concretas de formatos
- **Crema** — Implementaciones concretas de potenciales/restricciones

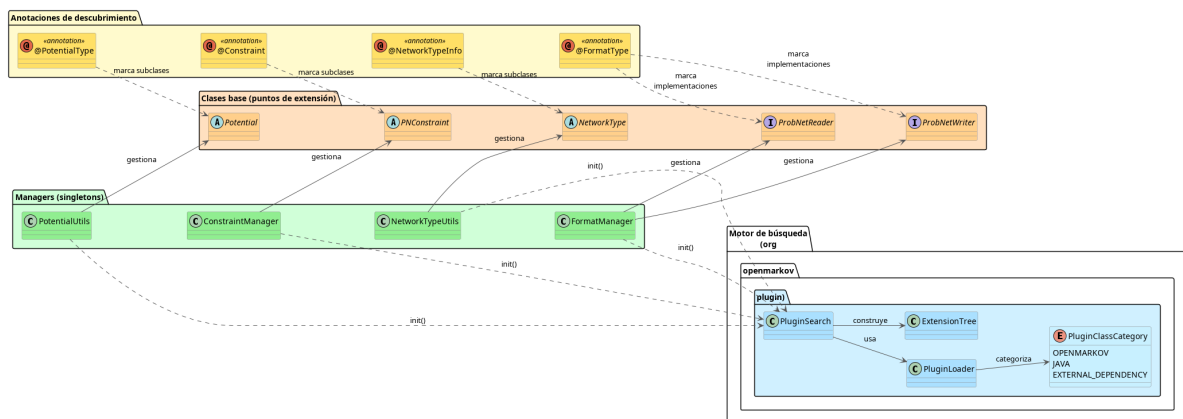


Figura 1: Vista general del sistema de plugins. De izquierda a derecha: anotaciones de descubrimiento, clases base, managers y motor de búsqueda.

§3 Motor de búsqueda de plugins

El paquete `org.openmarkov.plugin` proporciona la infraestructura de descubrimiento. Es independiente del dominio y reutilizable en cualquier módulo de OpenMarkov. Contiene tres clases principales y un enum.

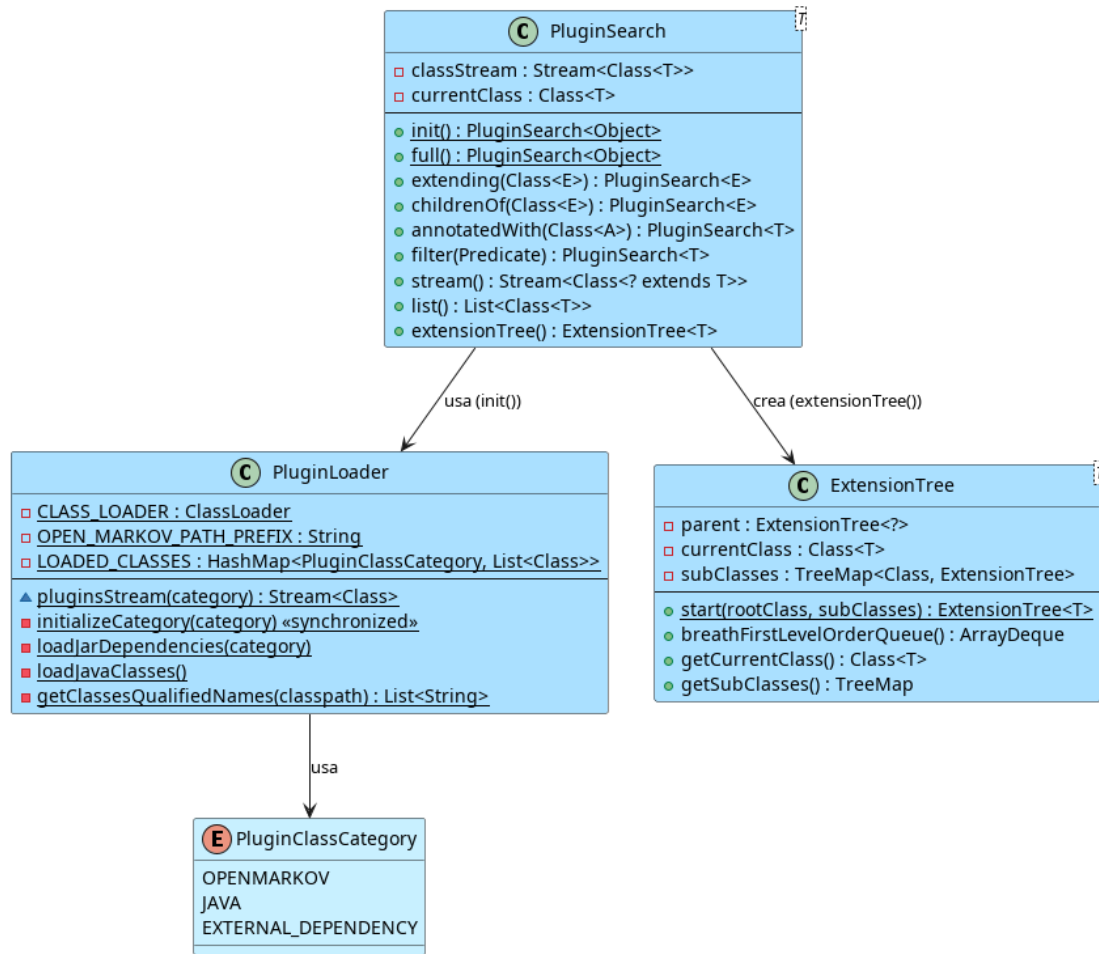


Figura 2: Motor de búsqueda: PluginSearch, PluginLoader y ExtensionTree

3.1 PluginClassCategory

Enumera las fuentes de clases que el motor sabe escanear:

OPENMARKOV Clases cuyo nombre de paquete comienza por `org.openmarkov`. Es la categoría usada por todos los managers del núcleo. El escaneo se realiza mediante **ClassGraph** sobre el classpath completo.

JAVA Clases del JDK, accedidas mediante el sistema de archivos virtual `jrt:/`.

EXTERNAL_DEPENDENCY Resto de clases del classpath (librerías de terceros, no OpenMarkov).

3.2 PluginLoader

Implementación interna (visibilidad de paquete). Responsabilidades:

1. **Enumeración:** usa `ClassGraph` para obtener las rutas de todos los JARs y directorios del classpath y extrae los nombres de clase de cada archivo `.class`.
2. **Carga en paralelo:** carga cada clase con `ClassLoader.getSystemClassLoader()` usando un stream paralelo.

3. **Caché:** el resultado se guarda en `LOADED_CLASSES` (un `HashMap` estático). Las categorías se inicializan de forma diferida y sincronizada: si dos threads solicitan la misma categoría simultáneamente, solo uno realiza el escaneo.

3.3 PluginSearch

API fluida e inmutable construida sobre `Stream<Class<T>`. Cada método devuelve una nueva instancia de `PluginSearch`, permitiendo encadenar operaciones sin efectos secundarios.

Método	Descripción
<code>init()</code>	Punto de entrada; obtiene el stream de <code>OPENMARKOV</code>
<code>full()</code>	Combina todas las categorías
<code>extending(Class<E>)</code>	Filtra subclases/implementaciones de E
<code>childrenOf(Class<E>)</code>	Como <code>extending</code> pero excluye la clase raíz
<code>annotatedWith(Class<A>)</code>	Filtra clases con la anotación A
<code>filter(Predicate)</code>	Filtro genérico
<code>stream()</code>	Operación terminal: devuelve el stream resultante
<code>list()</code>	Operación terminal: recoge en <code>List</code>
<code>extensionTree()</code>	Operación terminal: construye un <code>ExtensionTree</code>

```

1 Stream<Class<? extends PNConstraint>> restricciones =
2     PluginSearch.init()
3         .annotatedWith(Constraint.class)
4         .childrenOf(PNConstraint.class)
5         .stream();

```

Listado 1: Ejemplo de `PluginSearch` encadenado para buscar restricciones

3.4 ExtensionTree

Estructura de árbol que refleja la jerarquía de herencia. Se construye a partir de una clase raíz y un `Stream` de sus subclases. El método `breathFirstLevelOrderQueue()` recorre el árbol en **anchura (BFS)**, garantizando que las clases más cercanas a la raíz se procesen antes que las hojas. Esto es crucial para `PotentialUtils`: cuando una clase abstracta lleva `@PotentialType` y varias clases hoja la heredan, la clase más cercana a la raíz prevalece en el mapa de nombres.

§4 Anotaciones de descubrimiento y de contrato

Las anotaciones del sistema se dividen en dos grupos: las de *descubrimiento* (que marcan plugins) y las de *contrato* (que documentan requisitos de implementación).

Todas las anotaciones de descubrimiento comparten dos meta-anotaciones de Java:

- `@Retention(RetentionPolicy.RUNTIME)`: indica que la anotación debe estar disponible en tiempo de ejecución (y no solo en tiempo de compilación). Esto permite que `PluginSearch` la lea mediante reflexión durante el escaneo del classpath.

- `@Target(ElementType.TYPE)`: indica que la anotación solo puede aplicarse a declaraciones de tipo (clases, interfaces, enumeraciones y anotaciones). No puede aplicarse a métodos, campos ni parámetros.

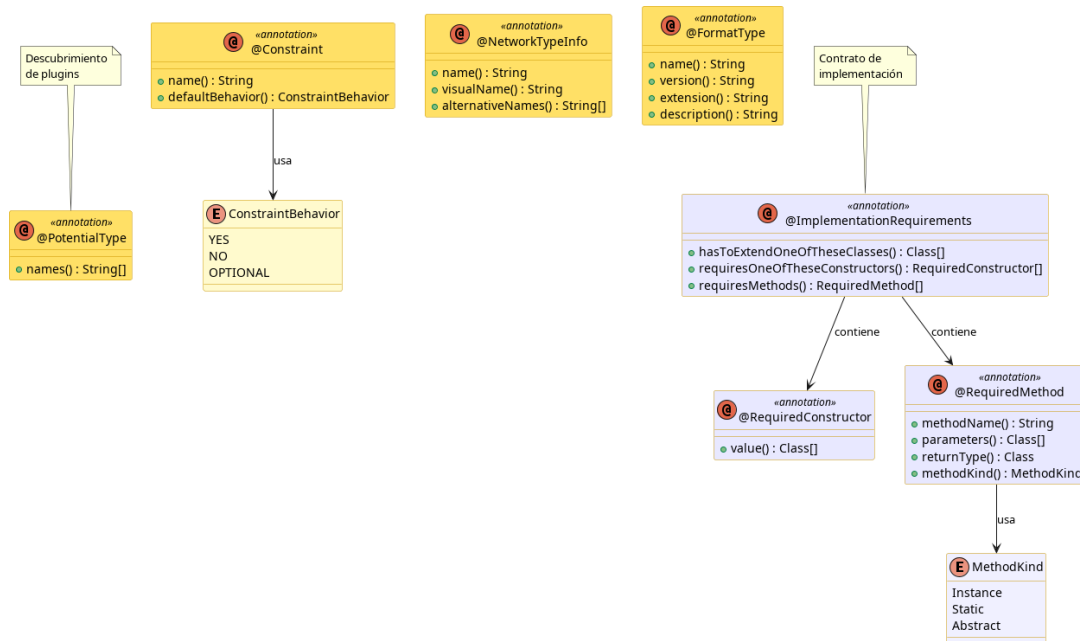


Figura 3: Todas las anotaciones del sistema de plugins

4.1 Anotaciones de descubrimiento

4.1.1 @PotentialType

Marca las subclases de `Potential` que deben ser descubiertas. El atributo `names` acepta un array para permitir múltiples alias del mismo tipo, útil para mantener compatibilidad con nombres históricos de archivos XML:

```

1 @PotentialType(names = {"Table", "ProbTable"})
2 public class TablePotential extends AbstractIndexedPotential {
3     ... }
4
5 @PotentialType(names = "Uniform")
6 public class UniformPotential extends Potential { ... }
7
8 @PotentialType(names = "ICIModel") // heredada por MaxPotential
9                                     y MinPotential
10 public abstract class ICIPotential extends Potential { ... }
  
```

Listado 2: Uso de `@PotentialType`

4.1.2 @Constraint

Marca las subclases de `PNConstraint`. El atributo `defaultBehavior` determina el comportamiento global de la restricción antes de que los tipos de red lo sobrescriban:

YES Activa en todos los tipos de red por defecto.

NO Inactiva por defecto; un tipo de red puede activarla.

OPTIONAL Inactiva por defecto, pero puede activarla el usuario.

4.1.3 @NetworkTypeInfo

Proporciona el nombre técnico (usado en ficheros XML), el nombre visual (mostrado en la GUI) y nombres alternativos para compatibilidad con versiones anteriores.

4.1.4 @FormatType

Incluye la extensión de archivo y la versión del formato para la resolución automática del lector/escritor. Permite registrar múltiples versiones del mismo formato (por ejemplo, pgmx 0.1 y pgmx 0.2).

4.2 Anotaciones de contrato

4.2.1 @ImplementationRequirements

Documenta qué constructores y métodos deben existir en *todas* las subclases concretas. **No impone restricciones en tiempo de ejecución:** la verificación se realiza en los tests de integración.

```

1 @ImplementationRequirements(
2     requiresOneOfTheseConstructors = {
3         @RequiredConstructor({List.class, CycleLength.class}),
4         // DBN
5         @RequiredConstructor({List.class, PotentialRole.class}),
6         // est ndar
7         @RequiredConstructor(List.class),
8         // m nimo
9         @RequiredConstructor(SelfClass.class)
10        // copy ctor
11    },
12    requiresMethods = @RequiredMethod(
13        methodKind = RequiredMethod.MethodKind.Static,
14        methodName = "validate",
15        returnType = Boolean.class,
16        parameters = {Node.class, List.class, PotentialRole.class}
17    )
18 )
19 public abstract class Potential implements Localizable { ... }

```

Listado 3: @ImplementationRequirements en Potential

La clase marcador `SelfClass` representa «el tipo de la propia subclase», exigiendo un constructor copia de la forma `SubTipo(SubTipo other)`.

Para `PNConstraint` el requisito es más sencillo: solo exige un constructor sin argumentos, necesario para que `ConstraintManager` las instancie por reflexión:

```

1 @ImplementationRequirements(
2     requiresOneOfTheseConstructors = @RequiredConstructor({}) //
3     sin args

```

```

3 )
4 public abstract class PNConstraint { ... }

```

Listado 4: @ImplementationRequirements en PNConstraint

§5 Managers

Un **manager** es un singleton que actúa como punto de acceso centralizado a una categoría de plugins. Sus responsabilidades son tres: (1) descubrir todas las clases de plugins disponibles al arrancar, usando `PluginSearch`; (2) mantener un registro interno (mapa de nombres a clases o lista de instancias) para que las búsquedas posteriores sean eficientes; y (3) exponer una API de alto nivel que el código de dominio puede usar sin conocer los detalles del sistema de reflexión subyacente.

Todos los managers se inicializan de forma diferida la primera vez que se accede a ellos, y el resultado del descubrimiento se cachea de forma permanente durante la vida de la JVM.

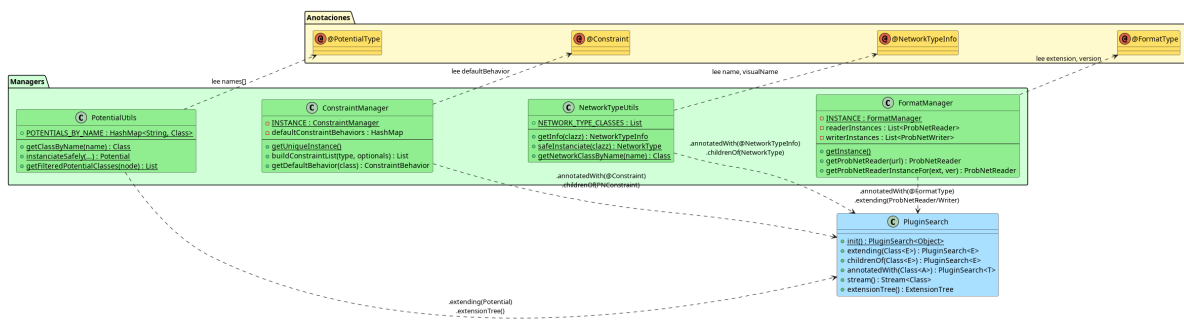


Figura 4: Los cuatro managers (columna central) y sus relaciones con el motor de búsqueda (izquierda) y las anotaciones que leen (derecha).

5.1 PotentialUtils

`PotentialUtils` es el manager de potenciales. Su responsabilidad es mantener el catálogo completo de tipos de potencial disponibles, permitir su búsqueda por nombre, instanciarlos de forma segura mediante reflexión, y determinar qué tipos son compatibles con un nodo concreto. Es la clase que permite que el sistema (y la GUI) traten los potenciales como entidades intercambiables sin conocer sus clases concretas.

El mapa `POTENTIALS_BY_NAME` se construye en el inicializador estático recorriendo el árbol de extensión de `Potential` en amplitud (BFS). Esto garantiza que, si hay un conflicto de nombre entre clase padre e hija, la más cercana a la raíz prevalece.

El método `instanciateSafely` prueba constructores en este orden:

1. `(List<Variable>, CycleLength)` — para redes dinámicas (DBN).
2. `(List<Variable>, PotentialRole)` — constructor estándar.
3. `(List<Variable>)` — constructor mínimo.

El método `getFilteredPotentialClasses(Node)` usa reflexión para invocar el método estático `validate(Node, List<Variable>, PotentialRole)` de cada potencial y devuelve solo los compatibles con el nodo dado.

5.2 ConstraintManager

ConstraintManager es el manager de restricciones. Su responsabilidad es conocer el conjunto completo de restricciones disponibles en el sistema y, para cada tipo de red concreto, construir la lista activa de restricciones que deben verificarse. Actúa como mediador entre el comportamiento global por defecto de cada restricción y las personalizaciones que cada tipo de red puede introducir.

La lógica de `buildConstraintList` opera en dos fases:

1. Recoge todas las restricciones con `defaultBehavior == YES` (y también las `OPTIONAL` si el parámetro `includeOptionals` es `true`).
2. Aplica las *sobrescrituras* del tipo de red concreto (`NetworkType.getOverwrittenConstraints()`): si una restricción está marcada `YES` en el tipo, se añade; si está marcada `NO`, se elimina de la lista.

```

1 public List<PNConstraint> buildConstraintList(NetworkType type,
2     boolean optionals) {
3     List<PNConstraint> result = new ArrayList<>();
4     // Fase 1: comportamientos por defecto globales
5     for (var entry : defaultConstraintBehaviors.entrySet()) {
6         if (entry.getValue() == YES || (optionals && entry.
7             getValue() == OPTIONAL))
8             result.add(instantiate(entry.getKey()));
9     }
10    // Fase 2: sobrescrituras del tipo de red
11    for (var entry : type.getOverwrittenConstraints().entrySet())
12    {
13        if (entry.getValue() == YES)
14            result.add(instantiate(entry.getKey()));
15        else if (entry.getValue() == NO)
16            result.removeIf(c -> c.getClass() == entry.getKey());
17    }
18    return result;
19 }

```

Listado 5: Lógica simplificada de `ConstraintManager.buildConstraintList`

5.3 NetworkTypeUtils

NetworkTypeUtils es el manager de tipos de red. Su responsabilidad es mantener el catálogo de todos los tipos de red disponibles en el sistema y proporcionar acceso a ellos por nombre. Es el componente que permite a los parsers de archivos y al código de dominio obtener la instancia singleton correcta de un tipo de red a partir de su nombre textual (por ejemplo, al cargar un archivo XML que declara `type="BayesianNetwork"`).

Todos los tipos de red son singletons. `NetworkTypeUtils.safeInstantiate` los obtiene llamando al método estático `getUniqueInstance()` por reflexión, evitando la instanciación duplicada.

La búsqueda por nombre prueba primero el atributo `name` de la anotación y, si no hay coincidencia, prueba los `alternativeNames`. Esto garantiza compatibilidad con archivos guardados con versiones anteriores de OpenMarkov.

5.4 FormatManager

FormatManager es el manager de formatos de archivo. Su responsabilidad es descubrir todos los lectores (**ProbNetReader**) y escritores (**ProbNetWriter**) disponibles, instanciarlos al arrancar y proporcionar el reader o writer correcto para cualquier archivo que se quiera abrir o guardar.

A diferencia de los otros managers, **FormatManager** **instancia inmediatamente** todos los plugins descubiertos en su constructor privado. El motivo es que la operación de abrir un archivo debe ser lo más rápida posible; pagar el coste de instanciación al arranque es preferible a pagarlo en cada apertura.

La resolución del reader para un archivo concreto combina extensión y versión:

1. Extrae la extensión del nombre de archivo.
2. Para formatos distintos de `.elv`, lee la versión del propio contenido XML del archivo.
3. Busca el reader cuya anotación `@FormatType` coincide con ambos valores (usando `startsWith` para la versión).

§6 Subsistemas de plugins

En esta sección se documenta cada categoría de plugin en detalle: las clases base que definen su contrato, las implementaciones concretas más relevantes y el comportamiento específico de su sistema de descubrimiento. Las cinco subsecciones siguientes corresponden a las cuatro categorías de plugin más las clases base que las sustentan.

6.1 Clases base de los puntos de extensión

Las clases base definen el contrato que todo plugin debe cumplir: qué métodos deben implementarse, qué constructor se exige y qué comportamiento estático se espera. Son las únicas clases del núcleo que conocen los tipos de plugin de forma abstracta; el resto del sistema opera siempre a través de los managers, nunca directamente sobre las implementaciones concretas.

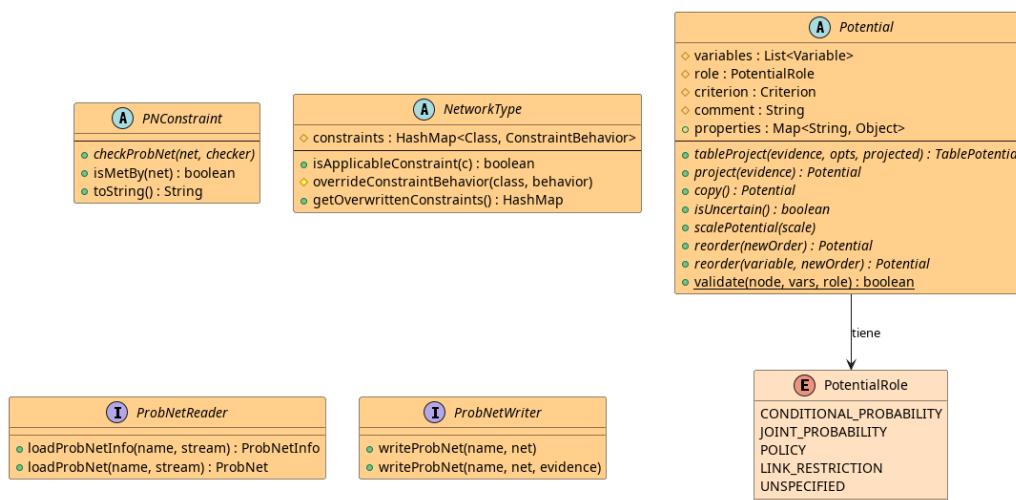


Figura 5: Jerarquía de las clases base que actúan como puntos de extensión

La tabla siguiente resume las obligaciones de cada clase base:

Clase base	Contrato para subclases
Potential	7 métodos abstractos + <code>static validate()</code>
PNConstraint	<code>checkProbNet()</code> + constructor sin args
NetworkType	<code>static getUniqueInstance()</code> + <code>@NetworkTypeInfo</code>
ProbNetReader	<code>loadProbNetInfo()</code> + <code>loadProbNet()</code> + ctor sin args
ProbNetWriter	<code>writeProbNet()</code> (2 sobrecargas) + ctor sin args

6.2 Subsistema de Potenciales

Los potenciales son el tipo de plugin más rico del sistema. Representan cualquier función matemática que mapea configuraciones de variables a valores numéricos: tablas de probabilidad condicional (CPTs), distribuciones paramétricas, formas canónicas como noisy-OR, árboles de decisión, etc. La gran variedad de tipos disponibles y la necesidad de filtrarlos por compatibilidad con el nodo hace de `PotentialUtils` el manager más complejo del sistema.

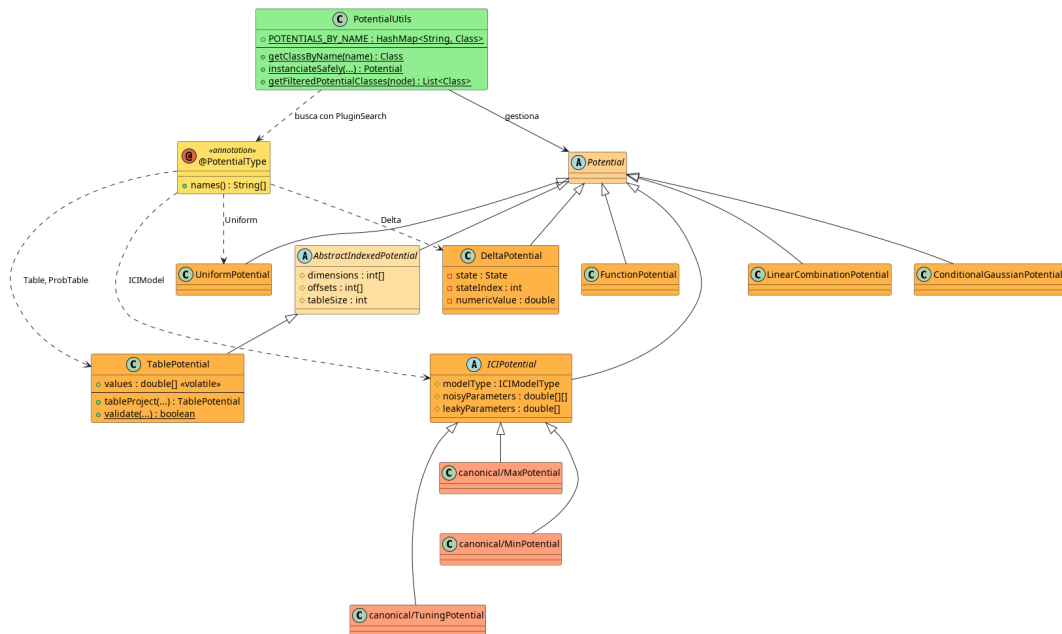


Figura 6: Subsistema de Potenciales: anotación, manager y jerarquía de implementaciones

6.2.1 Descubrimiento y herencia de la anotación

El inicializador estático de `PotentialUtils` usa `.extending(Potential.class)` en lugar de `.annotatedWith(@PotentialType)`. Este detalle es deliberado: permite que una clase abstracta intermedia (como `ICIPotential`) lleve la anotación `@PotentialType(names = "ICIModel")` y sus hijos concretos (`MaxPotential`, `MinPotential`) la *hereden* sin necesidad de repetirla.

```

1 static {
2     var byLevel = PluginSearch.init()
3     .extending(Potential.class) // hereda la anotación de
    clases padre

```

```

4      .extensionTree()
5      .breathFirstLevelOrderQueue()
6      .stream()
7      .map(ExtensionTree::getCurrentClass)
8      .toList();
9
10     byLevel.forEach(cls ->
11         PotentialUtils.getNames(cls).forEach(name ->
12             potentialsByName.put(name, cls))); // registra
13         todos los alias
14     POTENTIALS_BY_NAME = potentialsByName;
15 }

```

Listado 6: Descubrimiento de potenciales en el inicializador estático de PotentialUtils

6.2.2 Filtrado por compatibilidad

Cuando la GUI necesita saber qué potenciales son válidos para un nodo concreto, invoca `getFilteredPotentialClasses(node)`. Este método llama por reflexión al método estático `validate(Node, List<Variable>, PotentialRole)` de cada potencial. La implementación por defecto en `Potential` devuelve `true`; las subclases la sobrescriben. Por ejemplo, `TablePotential.validate` rechaza variables de tipo `NUMERIC`.

6.2.3 Indexación interna de TablePotential

`TablePotential` almacena la tabla en un array plano `double[] values`. La posición de una configuración $(x_0, x_1, \dots)$ es:

$$\text{pos} = x_0 \cdot \text{offsets}[0] + x_1 \cdot \text{offsets}[1] + \dots$$

donde `offsets[0]=1` y `offsets[i] = dimensions[i-1] * offsets[i-1]`. Esta indexación lexicográfica permite un acceso eficiente por indexación directa.

6.3 Subsistema de Restricciones

Las restricciones son reglas estructurales que una red debe cumplir en todo momento. Se verifican antes de ejecutar cada edición (`$PNEdit`) para garantizar que ninguna operación deja la red en un estado inválido. Son instancias sin estado (o con estado mínimo inmutable) que el sistema puede crear, aplicar y descartar sin efectos secundarios. El método `checkProbNet` añade excepciones al `ConstraintChecker` en lugar de lanzarlas directamente, lo que permite acumular varias violaciones y reportarlas juntas en un único error.

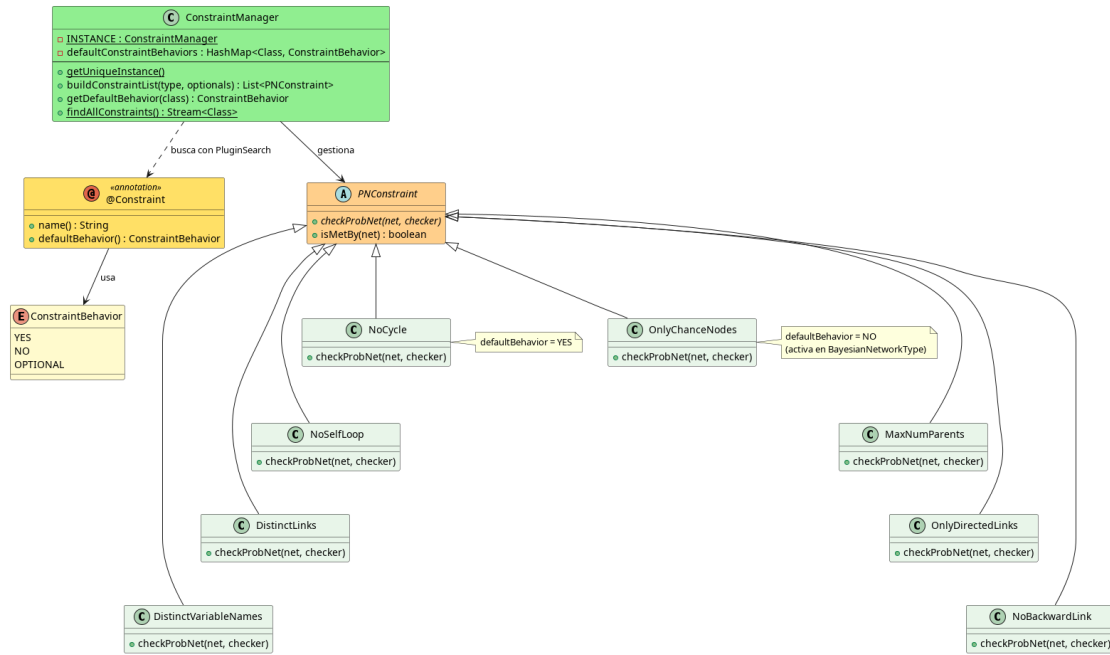


Figura 7: Subsistema de Restricciones: anotación, manager e implementaciones en dos columnas

6.3.1 Interacción con el sistema de ediciones

PNConstraint implementa PNEditListener. Antes de ejecutar una edición, esta llama a `checkConstraintsWillBeMet(checker)`, que añade al `ConstraintChecker` solo las violaciones relevantes a la operación que va a realizar, sin tener que reverificar toda la red desde cero.

6.4 Subsistema de Tipos de Red

Los tipos de red son el mecanismo por el cual OpenMarkov soporta distintos formalismos probabilísticos (redes bayesianas, diagramas de influencia, MDPs, etc.) con un único núcleo común. Cada tipo de red personaliza el conjunto de restricciones activas, determinando qué operaciones son válidas sobre ella. Son singletons inmutables porque su función es puramente declarativa: no almacenan datos de la red, solo sus reglas de validez.

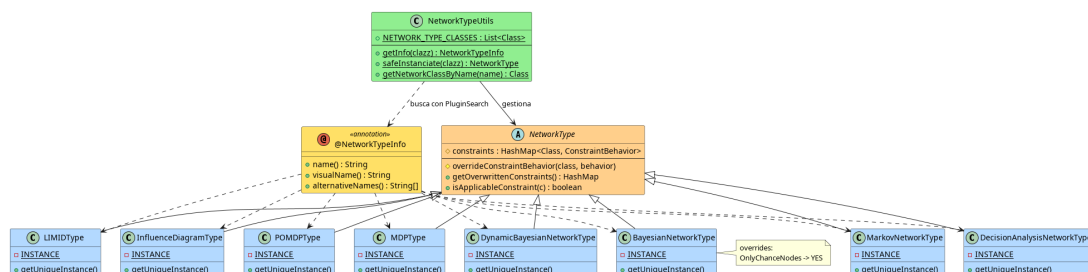


Figura 8: Subsistema de Tipos de Red: anotación, manager e implementaciones

```

1 @NetworkTypeInfo(name = "BayesianNetwork", visualName = "Bayesian
  Network")
2 public final class BayesianNetworkType extends NetworkType {

```

```
3     private static final BayesianNetworkType INSTANCE = new
        BayesianNetworkType();
4
5     private BayesianNetworkType() {
6         super();
7         // Activa una restricci n que por defecto global es NO
8         overrideConstraintBehavior(OnlyChanceNodes.class,
            ConstraintBehavior.YES);
9     }
10
11     public static synchronized BayesianNetworkType
        getUniqueInstance() {
12         return INSTANCE;
13     }
14 }
```

Listado 7: Estructura típica de un tipo de red (BayesianNetworkType)

6.5 Subsistema de Formatos de Archivo

Los plugins de formato son la puerta de entrada y salida del sistema: permiten leer redes probabilísticas desde archivos en disco y guardarlas de vuelta. Están diseñados para que añadir soporte a un nuevo formato (o una nueva versión de uno existente) no requiera modificar ningún código del núcleo; basta con añadir una nueva clase anotada con `@FormatType`.

A diferencia de los otros subsistemas, los plugins de formato se instancian de forma *inmediata* al arrancar: el **FormatManager** crea una instancia de cada reader y cada writer en su constructor privado. Esto garantiza que cualquier error de configuración (clase no encontrada, constructor ausente) se detecte durante el arranque y no al intentar abrir un archivo.

Un mismo módulo puede registrar tanto un reader como un writer para el mismo formato (extensión y versión) anotando clases separadas. El **FormatManager** los mantiene en listas distintas y los resuelve independientemente.

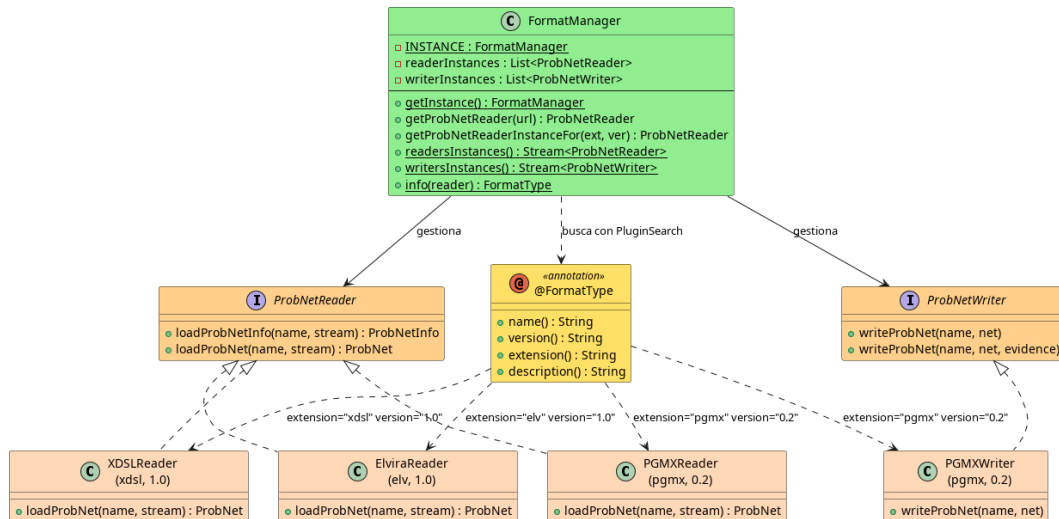


Figura 9: Subsistema de Formatos de Archivo: anotación, manager, interfaces y ejemplos de implementación

La resolución del reader para un archivo concreto combina tres datos: extensión del archivo, versión leída del contenido XML y la lista de readers disponibles. El método `getProbNetReaderInstanceFor(ext, version)` busca el primero cuya anotación `@FormatType` coincida con ambos valores, usando `startsWith` para la versión (por ejemplo, "0" coincide con "0.2").

§7 Flujos de descubrimiento: diagramas de secuencia

Los diagramas de secuencia siguientes muestran el flujo completo de descubrimiento para tres categorías de plugins: potenciales (el más complejo, con árbol de herencia y mapa de nombres), restricciones (con la doble fase de comportamientos globales y sobrescrituras por tipo de red) y formatos (con instanciación inmediata al arranque). En los tres casos, el flujo sigue el mismo patrón general: la primera referencia al manager dispara el descubrimiento vía `PluginSearch` y `PluginLoader`; el resultado se cachea; las llamadas posteriores son simples consultas al caché.

7.1 Descubrimiento de potenciales

El diagrama de la figura 7.1 muestra el flujo completo desde que la JVM referencia por primera vez a `PotentialUtils` (disparando el inicializador estático) hasta que el código externo obtiene una instancia de potencial. Hay dos fases bien diferenciadas: la *fase de construcción del catálogo* (carga del classpath, filtrado, BFS del árbol de herencia y registro en el mapa) y la *fase de uso* (búsqueda en el mapa e instanciación por reflexión). La fase de uso es $O(1)$ gracias al mapa preconstruido.

Un punto clave del flujo es que `PluginSearch` usa `.extending(Potential.class)` y no `.annotatedWith(@PotentialType)`, lo que garantiza que las clases que heredan la anotación de una clase padre sean también descubiertas.

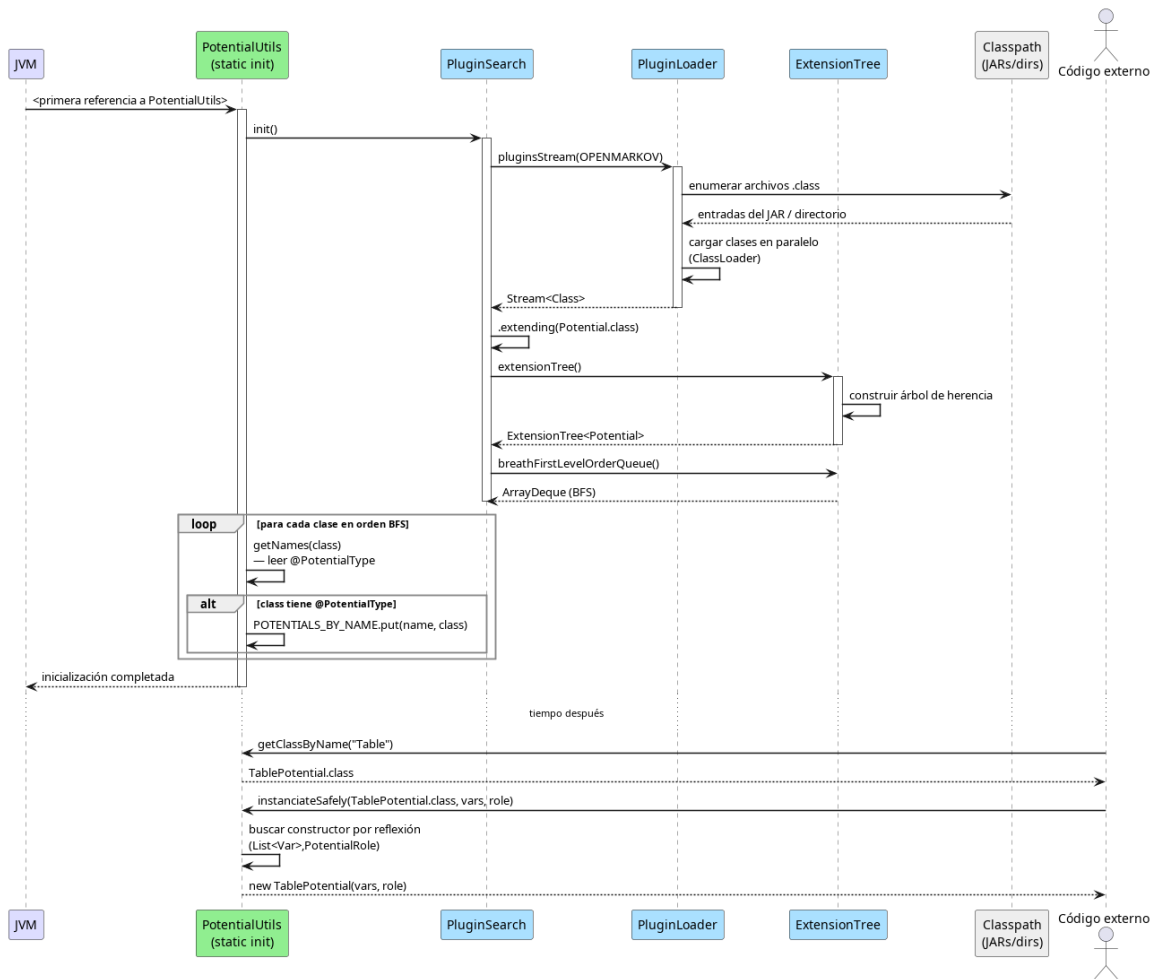


Figura 10: Secuencia de descubrimiento e instanciación de potenciales

7.2 Construcción de la lista de restricciones

El diagrama de la figura 7.2 muestra cómo la construcción de un **ProbNet** con tipo **BayesianNetworkType** resulta en una lista de restricciones activas. El flujo tiene dos momentos temporalmente separados: el *arranque de la JVM*, en el que **ConstraintManager** se inicializa y descubre todas las restricciones disponibles; y la *creación de una red* concreta, en la que **buildConstraintList** aplica los comportamientos por defecto y luego las sobrescrituras del tipo de red.

El efecto de la doble fase se ve claramente con **OnlyChanceNodes**: su **defaultBehavior** global es **NO**, por lo que no entra en la lista inicial; pero **BayesianNetworkType** la sobrescribe con **YES**, de modo que se añade en la segunda fase.

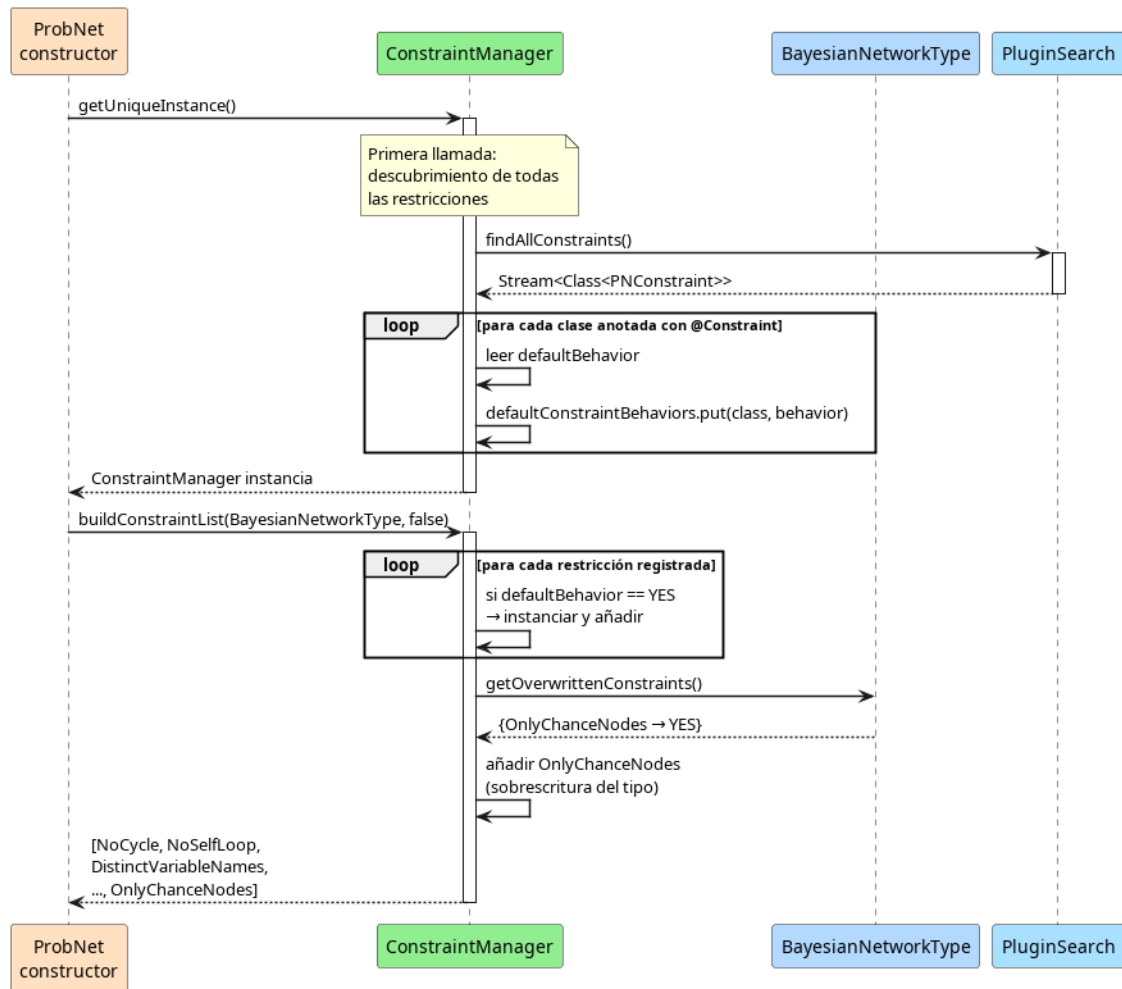


Figura 11: Secuencia de construcción de la lista de restricciones para una red bayesiana

7.3 Resolución de formato al abrir un archivo

El diagrama de la figura 7.3 ilustra el flujo del **FormatManager**. A diferencia de los otros dos, aquí el arranque es más costoso: el manager instancia inmediatamente *todos* los readers y writers disponibles. La recompensa llega en el momento de abrir un archivo: la resolución del reader es una simple búsqueda sobre una lista de instancias ya creadas, sin reflexión ni instanciación en ese momento.

El diagrama también muestra la lectura de la versión del XML como paso intermedio entre la extracción de la extensión y la búsqueda del reader. Este paso es necesario porque el mismo formato puede tener múltiples versiones con parsers distintos (por ejemplo, pgmx 0.1 y pgmx 0.2).

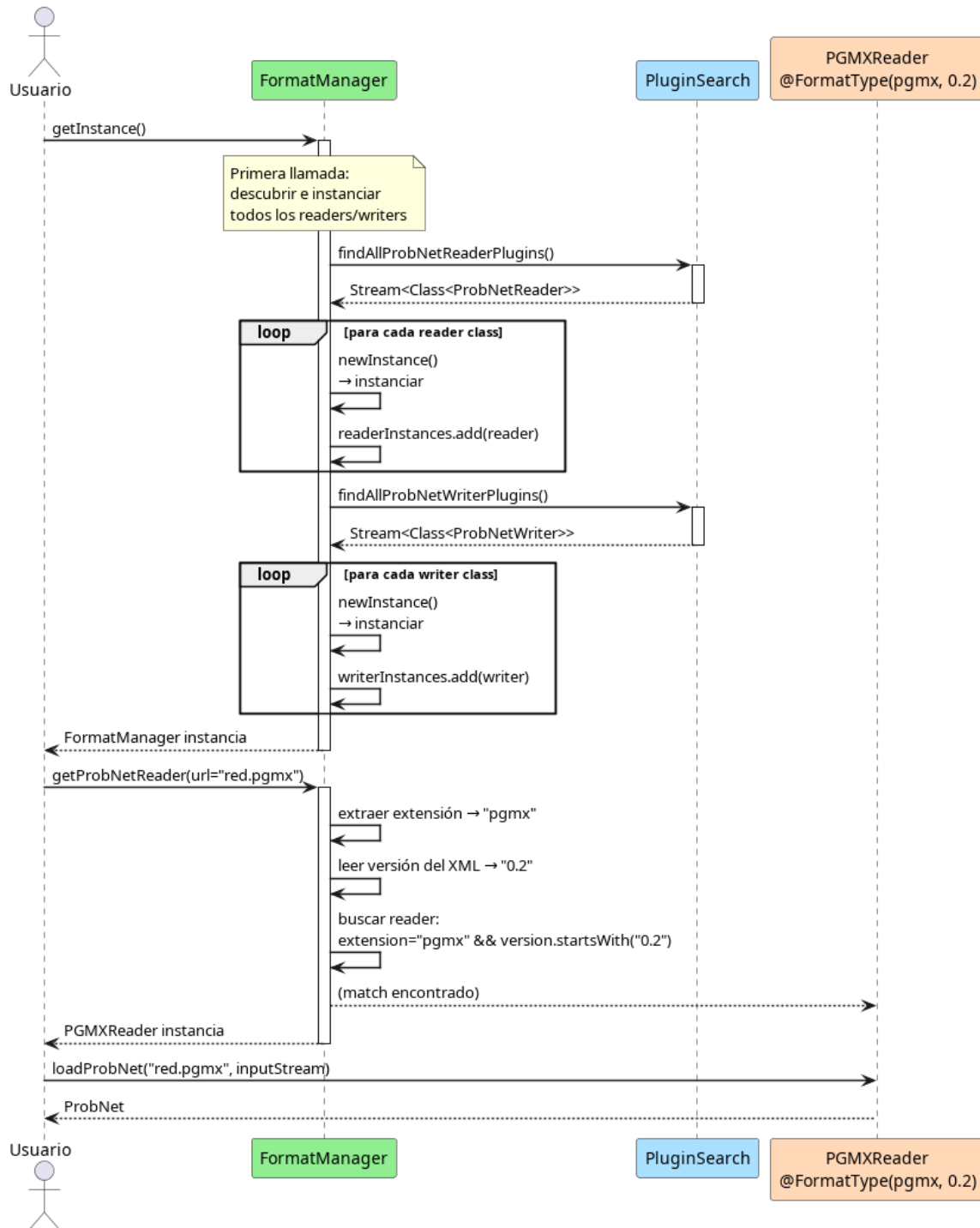


Figura 12: Secuencia de resolución y uso del lector de formato

§8 Cómo añadir un nuevo plugin

El sistema está diseñado para que añadir un nuevo plugin requiera únicamente:

1. Crear la clase en cualquier módulo cuyo JAR esté en el classpath.
2. Anotar con la anotación apropiada.

3. Cumplir el contrato de @ImplementationRequirements.

No es necesario modificar ninguna clase existente del núcleo.

8.1 Nuevo Potencial

```

1 @PotentialType(names = {"MiPotencial", "MyPotential"})
2 public class MiPotencial extends Potential {
3
4     // Constructor estandar (requerido por
5     // @ImplementationRequirements)
6     public MiPotencial(List<Variable> variables, PotentialRole
7     role) {
8         super(variables, role);
9     }
10
11     // Constructor copia (requerido: SelfClass en
12     // @ImplementationRequirements)
13     public MiPotencial(MiPotencial other) { super(other); }
14
15     // Filtro de compatibilidad (requerido: @RequiredMethod
16     // validate)
17     public static boolean validate(Node node, List<Variable>
18     variables,
19     PotentialRole role) {
20         return variables.stream()
21             .allMatch(v -> v.getVariableType() == VariableType.
22             FINITE_STATES);
23     }
24
25     @Override
26     public TablePotential tableProject(EvidenceCase evidence,
27     InferenceOptions options,
28     List<TablePotential>
29     projected) { ... }
30
31     @Override public Potential project(EvidenceCase evidence) {
32     ... }
33
34     @Override public Potential copy() { return new MiPotencial(
35     this); }
36
37     @Override public boolean isUncertain() { return false; }
38
39     @Override public void scalePotential(double scale) { ... }
40
41     @Override public Potential reorder(List<Variable> newOrder) {
42     ... }
43
44     @Override public Potential reorder(Variable v, State[]
45     newOrder) { ... }
46 }
47
48 // Acceso: PotentialUtils.getClassByName("MiPotencial")

```

Listado 8: Implementación de un nuevo potencial

8.2 Nueva Restricción

```

1 @Constraint(name = "MiRestriccion", defaultBehavior =
   ConstraintBehavior.YES)
2 public class MiRestriccion extends PNConstraint {
3
4     // Constructor sin argumentos (requerido por
       @ImplementationRequirements)
5     public MiRestriccion() { super(); }
6
7     @Override
8     public void checkProbNet(ProbNet probNet, ConstraintChecker
       checker) {
9         for (Node node : probNet.getNodes()) {
10             if (condicionDeViolacion(node))
11                 checker.addException(new
                   ConstraintViolatedException.MiError(this, node)
                   );
12         }
13     }
14 }
15 // Un tipo de red puede desactivarla con:
16 //     overrideConstraintBehavior(MiRestriccion.class,
       ConstraintBehavior.NO);

```

Listado 9: Implementación de una nueva restricción

8.3 Nuevo Tipo de Red

```

1 @NetworkTypeInfo(
2     name = "MiRed",
3     visualName = "Mi Tipo de Red",
4     alternativeNames = {"MyNet"}
5 )
6 public final class MiRedType extends NetworkType {
7
8     private static final MiRedType INSTANCE = new MiRedType();
9
10    private MiRedType() {
11        super();
12        overrideConstraintBehavior(OnlyChanceNodes.class,
           ConstraintBehavior.YES);
13        overrideConstraintBehavior(NoBackwardLink.class,
           ConstraintBehavior.NO);
14    }
15
16    // Obligatorio: NetworkTypeUtils lo invoca por reflexi n
17    public static synchronized MiRedType getUniqueInstance() {
18        return INSTANCE; }
19 }

```

Listado 10: Implementación de un nuevo tipo de red

8.4 Nuevo Formato de Archivo

```

1 @FormatType(
2     name          = "MiFormatoReader",
3     version       = "1.0",
4     extension     = "mif",
5     description   = "MiFormato.description"    // clave i18n para la
        GUI
6 )
7 public class MiFormatoReader implements ProbNetReader {
8
9     public MiFormatoReader() { }    // constructor sin args
        obligatorio
10
11     @Override
12     public ProbNetInfo loadProbNetInfo(String netName,
        InputStream file)
13         throws IOException, ParseException {
14         return new ProbNetInfo(...);    // metadatos ligeros
15     }
16
17     @Override
18     public ProbNet loadProbNet(String netName, InputStream file)
19         throws IOException, ParseException {
20         ProbNet net = new ProbNet(MiRedType.getUniqueInstance());
21         // parseo completo...
22         return net;
23     }
24 }
25 // Acceso: FormatManager.getInstance().getProbNetReader(
        urlArchivoMif)

```

Listado 11: Implementación de un nuevo formato de archivo

§9 Mapa completo del sistema

La figura 9 muestra cómo todos los componentes se ensamblan en una vista única. El diagrama está organizado en dos franjas horizontales: en la superior aparecen el motor de búsqueda, los managers y el código de dominio (de izquierda a derecha según su papel en el flujo de descubrimiento); en la inferior aparece el contenido del classpath con todos los tipos de plugins, dispuestos horizontalmente para mostrar que son entidades independientes que conviven en el mismo espacio de clases.

Las flechas del motor al classpath reflejan la carga por `PluginLoader`; las flechas de los managers al motor reflejan las búsquedas por `PluginSearch`; y las flechas del dominio a los managers reflejan el uso en tiempo de ejecución (crear redes, validar ediciones, instanciar potenciales).

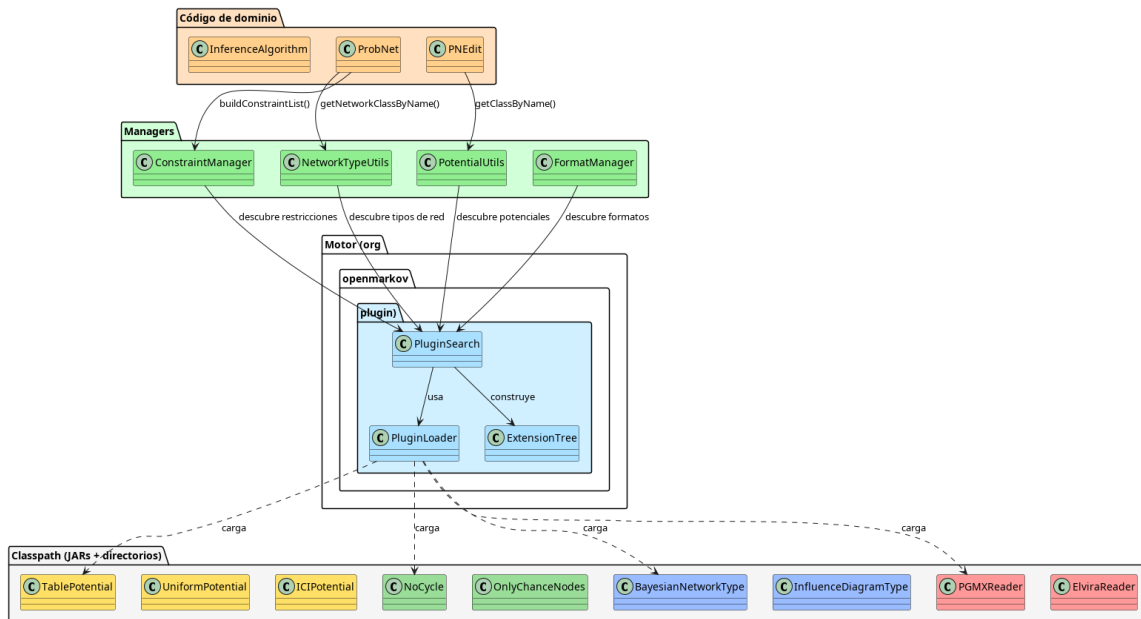


Figura 13: Mapa completo del sistema de plugins

§10 Consideraciones de rendimiento y ciclo de vida

10.1 Coste de inicialización

El escaneo del classpath por PluginLoader es la operación más costosa. Para mitigarlo:

- El escaneo se realiza **una sola vez** por categoría y el resultado se cachea en LOADED_CLASSES (mapa estático final).
- La carga de clases individuales se hace **en paralelo** (stream paralelo).
- Los mapas de nombres se construyen una sola vez en inicializadores estáticos.
- FormatManager paga el coste de instanciación al arranque para que las aperturas de archivo posteriores sean inmediatas.

10.2 Thread safety

- PluginLoader.initializeCategory está sincronizado.
- ConstraintManager y FormatManager son singletons con campo static final (garantía de la JVM).
- Los tipos de red tienen getUniqueInstance() sincronizado.

10.3 Extensibilidad en tiempo de ejecución

El sistema *no* soporta recarga dinámica de plugins en caliente. Para añadir un plugin es necesario reiniciar la JVM con el nuevo JAR en el classpath.

10.4 Resumen comparativo

Manager	Búsqueda Plugin- Search	Init	Instanciación	Caché
PotentialUtils	.extending(PotentialUtil)		Diferida por nombre	HashMap
ConstraintManager	.annotatedWith(@Constraint)		construir lista	HashMap
	.childrenOf(PNConstraint)			
NetworkTypeUtils	.annotatedWith(@NetworkTypeUtil)		getUniqueInstance()	List
	.childrenOf(NetworkType)			
FormatManager	.annotatedWith(@FormatType)		mediata	List
	.extending(Reader/Writer)			